

1. Screenshot of Footers (5%)

The screenshot displays the footer and two main forms: 'Sign Up' and 'Login'. The footer, located at the bottom of the page, contains the text '2024 NYCU DB HW3 112550200 鄭武謙. All rights reserved.' on a light purple background. The 'Sign Up' form on the left includes a title 'Sign Up', a 'Username:' label with an input field containing 'Enter a username', a 'Password:' label with an input field containing 'Enter a password', a green 'Sign Up' button, and a link 'Already have an account? [Back to Login](#)'. The 'Login' form on the right includes a title 'Login', a 'Username:' label with an input field containing 'Enter your username', a 'Password:' label with an input field containing 'Enter your password', a green 'Log In' button, and a link 'Don't have an account? [Sign Up](#)'. Both forms are set against a light purple background.

2. Screenshot and Explanation of Login Function (5%)

```
if result and result[0] == password: # Check if the password matches
    session['username'] = username
    return redirect("/welcome")
else:
    flash("Invalid username or password", "danger") # Flash error message

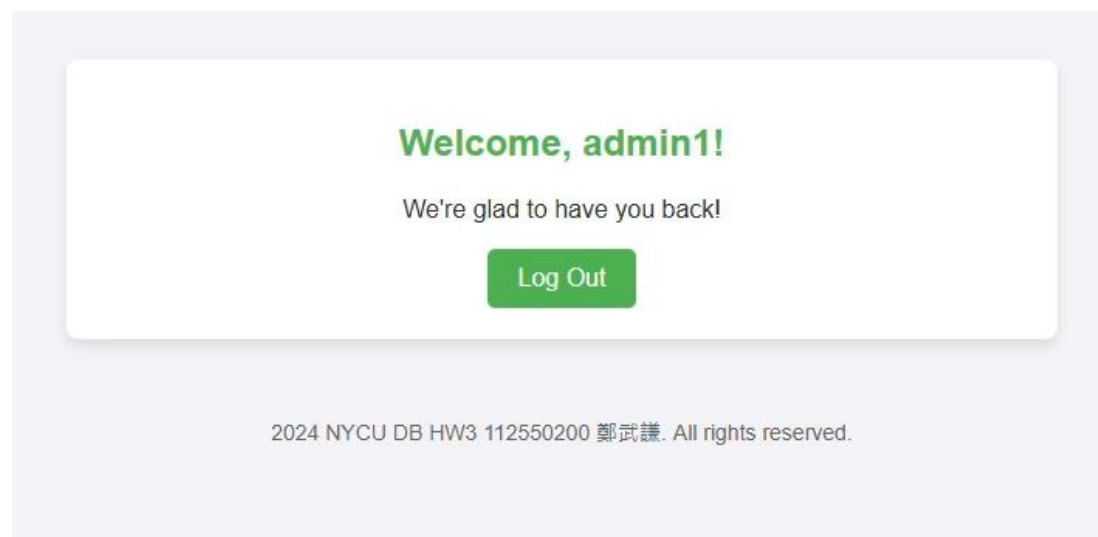
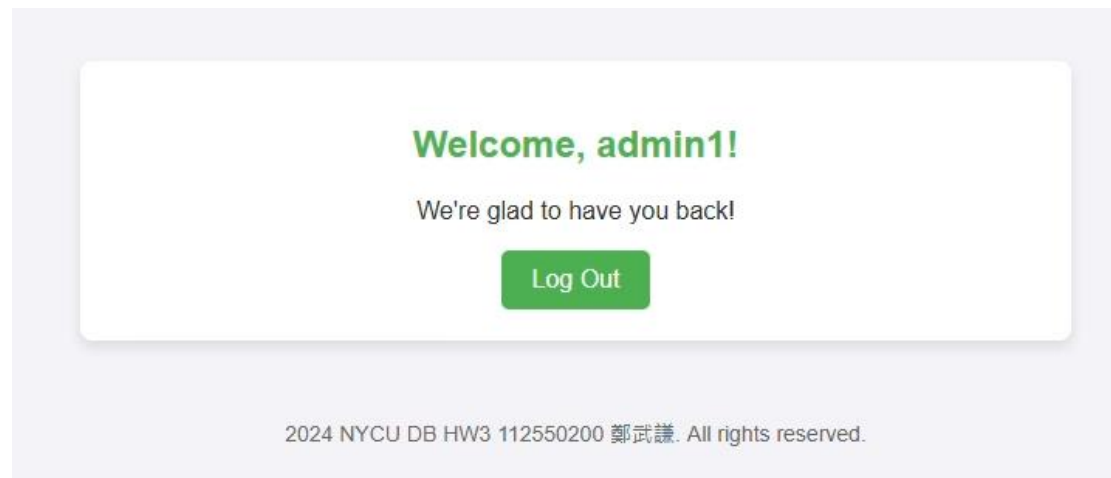
# Close the connection
cursor.close()
conn.close()
```

在實作上，我們將輸入進來的帳號密碼與資料庫進行比對，如果資料庫中有相應的帳號密碼，那就跳到 `welcome` 的頁面，反之，就利用 `flash` 顯示輸入的帳號密碼錯誤。

3. Screenshot and Explanation of Signup Function (5%)

```
# TODO # 3: Add the query to insert a new user into the database
try:
    # Insert new user into the database
    cursor.execute("INSERT INTO users (username, password) VALUES (%s, %s)", (username, password))
    conn.commit()
    flash("Account created successfully! Please log in.", "success")
    return redirect("/")
except mysql.connector.Error as err:
    flash(f"Error: {err}", "danger")
finally:
    cursor.close()
    conn.close()
return render_template("signup.html")
```

先試著將新用戶及其密碼插入資料庫裡，如果資料庫內沒有重複的用戶名稱，插入後利用 flash 顯示創建成功，接著導引回登入頁面，如果資料庫內有重複的用戶名稱則用 flash 顯示創建失敗。



4. Screenshot of Hashed Passwords (5%)

```
# Login Page
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form['username']
        password = request.form['password']

        # TODO # 4: Hash the password using SHA-256
        password = hashlib.sha256(password.encode()).hexdigest()
```

import hashlib 來使用 sha256 的函式，利用這個函式可以直接將密碼轉為 sha256 的格式。

接著在後續的 login 和 singup 來使用。

```
+-----+
| id | username | password |
+-----+
| 1 | admin   | 1234     |
| 3 | admin1  | 03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4 |
+-----+
2 rows in set (0.00 sec)
```

5. Screenshot and Explanation of SQL Injection Prevention (10%)

```
cursor.execute("SELECT password FROM users WHERE username = %s", (username,))  
result = cursor.fetchone() # fetchone() returns None if no record is found
```

為防止 SQL injection，我使用了參數代替，而非直接插入用戶的輸入到 SQL 裡。為做到這點，我使用 %s 並將輸入作為參數傳遞。

The screenshot shows a web application login interface. At the top, the word "Login" is displayed in green. Below it, there are two input fields: "Username:" and "Password:". The "Username:" field contains the text "admin' OR '1'='1", which is a common SQL injection payload. The "Password:" field contains a single dot ".". Below the input fields is a green "Log In" button. Underneath the button, there is a link that says "Don't have an account? Sign Up". At the bottom of the form, there is a red error message box that says "Invalid username or password".

2024 NYCU DB HW3 112550200 鄭武謙. All rights reserved.